

Table 1 Comprehensive tasks on lm-eval-harness (Gao et al., 2021). MAGICPIG significantly outperforms other methods with lower computation. The config (K, L) is a hyper-parameter of LSH for MAGICPIG or page size and ratio of selected pages for Quest (Tang et al., 2024). Cost₁, Cost₂ represents the cost for searching/sampling and sparse attention computation.

Methods	Config	GSM	COQA	MMLU	Avg.	Cost ₁	Cost ₂	Cost _{total} .
<i>Llama-2-7b-chat</i>	Full	22.4	75.8	49.2	49.1	0.00	1.00	1.00
MAGICPIG	(10,220)	17.3	76.4	48.6	47.4	0.00	0.04	0.04
MAGICPIG	(8,90)	18.7	75.0	47.9	47.2	0.00	0.08	0.08
Quest	(16,0.05)	13.0	69.4	41.4	41.3	0.06	0.05	0.11
Quest	(32,0.1)	15.7	70.2	44.0	43.3	0.03	0.10	0.13
<i>Llama-3.1-8B-Instruct</i>	Full	77.6	78.5	65.2	73.7	0.00	1.00	1.00
MAGICPIG	(10,220)	72.7	78.1	62.7	71.2	0.00	0.03	0.03
MAGICPIG	(8,90)	71.0	78.0	61.3	70.1	0.00	0.07	0.07
Quest	(16,0.05)	57.9	64.6	42.5	55.0	0.06	0.05	0.11
Quest	(32,0.1)	64.5	65.0	48.0	59.2	0.03	0.10	0.13

Table 2 Long context tasks on LongBench (Bai et al., 2023). MAGICPIG preserves high accuracy with low computation. Config and cost are defined as in Table 1. Code models are only evaluated by Repobench-P and LCC.

Methods	Config	QaS	RbP	LCC	PrE	TrC	TrQ	Avg.	Cost ₁	Cost ₂	Cost _{total} .
<i>Llama-3.1-8B-Instruct</i>	Full	44.9	52.1	66.8	100.0	71.3	91.8	71.2	0.00	1.00	1.00
MAGICPIG	(10,150)	43.2	50.2	64.4	100.0	71.3	92.2	70.3	0.00	0.02	0.02
MAGICPIG	(8,75)	43.5	50.4	67.0	100.0	71.7	91.7	70.7	0.00	0.05	0.05
Quest	(16,0.05)	45.7	49.7	64.9	100.0	71.7	91.5	70.6	0.06	0.05	0.11
Quest	(32,0.1)	44.4	50.5	65.1	100.0	71.3	91.6	70.5	0.03	0.10	0.13
<i>Code-Llama-13b-16K</i>	Full		58.5	74.7				66.6	0.00	1.00	1.00
MAGICPIG	(10,150)		56.9	74.0				65.5	0.00	0.03	0.03
Quest	(16,0.05)		56.4	74.4				65.4	0.06	0.05	0.11

the low computation budget regime, we also evaluate Quest with page size 32 and 64 and make sure at least one page is selected in every test example. The initial 4 tokens and local 64 (for LongBench (Bai et al., 2023) and RULER (Hsieh et al., 2024)) or 24 (for lm-eval-harness (Gao et al., 2021)) tokens as well as layer- $\{0, 16\}$ are statically preserved. We do not use the theoretical transformations in Equation (8) in our implementations, as we do not find them to contribute to accuracy improvements.

Cost. The cost for the attention approximation consists of two parts: Cost₁ is the sampling/search cost to obtain S in Equation (11), Cost₂ is the attention computation cost, see Equation (11). We report the ratio of the number of FLOPs compared to the full attention computation. For MAGICPIG, Cost₁ $\simeq 0$ and Cost₂ is empirically measured for different LSH hyper-parameters. For Quest with page size K , Cost₁ = $\frac{1}{K}$ and Cost₂ is controlled manually.

Analysis. From Tables 1 to 3, (1) MAGICPIG preserves high accuracy (degradation less than 2%) for all kinds of tasks, with a computation cost of 2% $\sim$ 5%. (2) Compared with Quest, which also shows reasonable performance on long context tasks, MAGICPIG also demonstrates good performance on tasks with moderate context sizes in lm-eval-harness (Gao et al., 2021), indicating a more robust performance in general serving. (3) With LSH sampling, which introduces an order of magnitude lower sampling/searching cost (Cost₁), MAGICPIG can achieve equivalent or better accuracy with only half of the computation cost.

5.2 MagicPIG Shows Impressive Efficiency across Various Hardware Settings

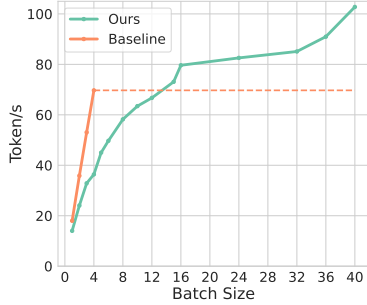
We show MAGICPIG can bring up to 5 $\times$ wall clock speed up and reduce GPU memory consumption on different models and hardware settings (A100, L20, RTX4090).

Setup. We evaluate our system performance on 3 serving settings. (1) 80GB GPU (A100) and 34B model (CodeLlama-34B) (Rozière et al., 2024) with 16K contexts; (2) 48GB GPU (L20) and 13B model (CodeLlama-13B) (Rozière et al., 2024) with 16K contexts; (3) 24GB GPU³ (e.g. RTX 4090) and 8B model (Llama-3.1-

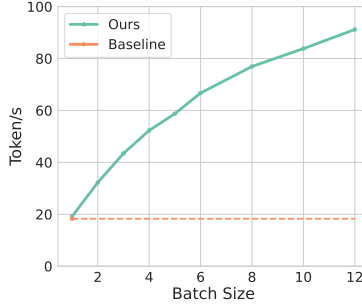
³We simulate 24GB GPU by setting memory limit with L20. As the bandwidth of L20 (864GB/s) is less than RTX 4090 (1TB/s), the real speed of our system should be slightly faster than the simulation.

Table 3 Synthesized tasks on RULER (Hsieh et al., 2024). MAGICPIG preserves high accuracy with low computation. Config and cost are defined as in Table 1.

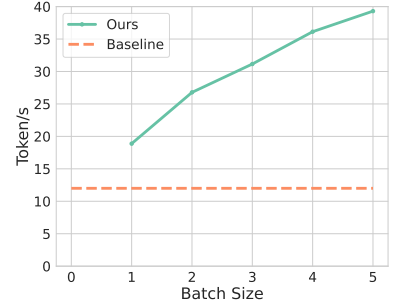
Methods	Config	16K	32K	64K	96K	Avg.	Cost ₁	Cost ₂	Cost _{total}
<i>Llama-3.1-8B-Instruct</i>	Full	94.2	91.5	86.1	83.0	88.7	0.00	1.00	1.00
MAGICPIG	(10,150)	91.8	88.9	84.8	80.0	86.4	0.00	0.02	0.02
MAGICPIG	(9,120)	93.4	90.6	84.7	81.5	87.6	0.00	0.04	0.04
MAGICPIG	(8,75)	92.9	90.2	84.9	81.7	87.4	0.00	0.05	0.05
Quest	(16,0.04)	86.3	85.4	81.9	74.9	82.1	0.06	0.04	0.10
Quest	(32,0.06)	84.3	84.0	80.1	74.4	80.7	0.03	0.06	0.09
Quest	(64,0.08)	85.2	84.3	77.0	74.2	80.2	0.02	0.08	0.10
<i>MegaBeam-Mistral-7B-512K</i>	Full	91.7	88.1	83.5	83.7	86.8	0.00	1.00	1.00
MAGICPIG	(10,150)	89.8	86.5	81.7	80.7	84.7	0.00	0.02	0.02
MAGICPIG	(9,120)	90.7	88.5	82.9	82.4	86.1	0.00	0.04	0.04
MAGICPIG	(8,75)	90.6	86.4	82.8	81.6	85.4	0.00	0.05	0.05
Quest	(16,0.04)	83.3	83.2	79.3	78.6	81.1	0.06	0.04	0.10
Quest	(32,0.06)	81.5	80.8	76.7	74.4	78.4	0.03	0.06	0.09
Quest	(64,0.08)	79.6	77.5	73.8	73.7	76.1	0.02	0.08	0.10
<i>Llama3-8B-Prolong-512K</i>	Full	93.5	90.8	85.1	83.5	88.2	0.00	1.00	1.00
MAGICPIG	(10,150)	88.0	86.4	81.3	78.8	83.6	0.00	0.02	0.02
MAGICPIG	(10,170)	89.0	88.7	82.8	80.0	85.1	0.00	0.025	0.025
MAGICPIG	(9,120)	91.4	88.2	82.4	80.4	85.6	0.00	0.04	0.04
MAGICPIG	(8,75)	91.4	88.6	83.1	80.5	85.9	0.00	0.05	0.05
Quest	(16,0.04)	84.9	83.7	78.7	78.6	81.5	0.06	0.04	0.10



(a) A100 with 34B model



(b) L20 with 13B model



(c) RTX 4090 with 8B model

Figure 8 We evaluate MAGICPIG on three serving scenarios. **Left:** A100 serves 34B model with 16K context. MAGICPIG achieves 1.5× throughput improvement. **Mid:** L20 serves 13B model with 16K context. MAGICPIG achieves 5.0× throughput improvement. **Right:** Simulated RTX 4090 serves 8B model with 96K context. MAGICPIG achieves a latency of 54ms in a single request serving and can improve the throughput of baseline by up to 3.3×. The dashed lines denote the highest throughput of baselines. With KV cache offloading, MAGICPIG can fit a much larger batch size compared with full attention on GPU, which contributes to the throughput improvement.

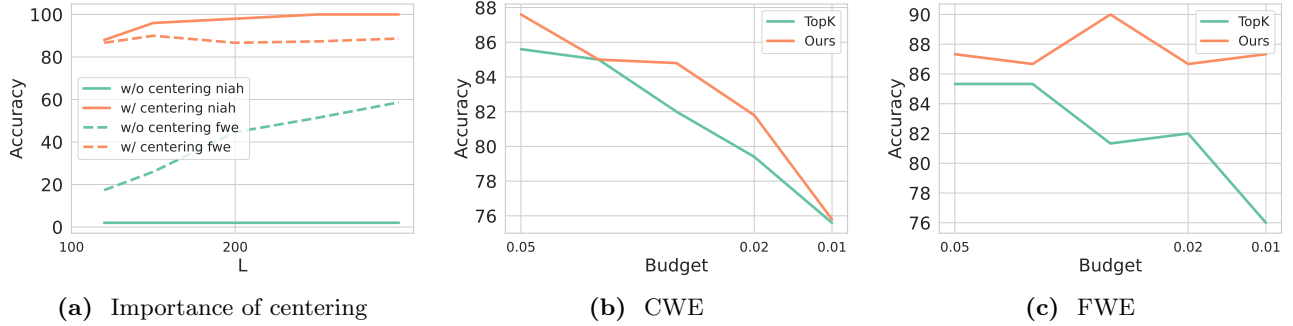


Figure 9 **Left:** Accuracy comparison for with and without centering. Here we fix K and vary L for the two settings. **Mid and Right:** Comparison between TopK attention and MAGICPIG. In the two aggregated tasks, sampling-based MAGICPIG can even beat the exact TopK attention. The experiments are done on RULER (Hsieh et al., 2024) with a 16K context size.

8B) (Dubey et al., 2024) with 96K contexts.

Baselines. Our baselines for (1) and (2) are full attention on GPU, and for (3) is full attention on CPU with theoretical estimated bandwidth. Our system’s GPU part is implemented in native Pytorch (Paszke et al., 2019) and the CPU part in FBGEMM (Khudia et al., 2021) in bfloat16 precision. Our CPU is Intel Platinum 8480+ for A100 and Intel 8563C for L20. In the last setting, the CPU bandwidth is estimated at 150GB/s, above the empirical bandwidth we measure when running a group query attention of size 4.

Analysis. In Figures 8a to 8c, we demonstrate (1) MAGICPIG significantly improves decoding throughput for all three scenarios (A100: 1.5 $\times$, L20: 5.0 $\times$, RTX 4090: 3.3 $\times$) and can achieve a latency of 54ms for single request generation with 96K context for RTX 4090. (2) With KV cache offloading, MAGICPIG can fit much larger batches than GPU full attention baselines (over 12 $\times$). The ablation study of decoding throughput with different LSH hyper-parameters is presented in Table 7.

5.3 Ablation Study

In this section, we empirically validate our two previous observations.

Centering is important for good performance. In Section 4.3, we use a translation to center the keys before applying LSH sampling. Empirical results show this to be important for downstream tasks as shown in Figure 9a. Without centering, the accuracy drops to almost zero in retrieval (NIAH) and degrades to 65% in FWE. We find almost no keys (less than 0.1%) can be sampled by the query without centering, as their orientation is almost opposite, as shown in Figure 2c.

Sampling goes beyond TopK. In Figures 9b and 9c, We compare the performance of MAGICPIG and TopK attention in two aggregated tasks (CWE, FWE) where TopK attention experiences significant performance degradation (Figure 1). MAGICPIG can even beat exact TopK attention in these two tasks by a margin up to 3% and 8% respectively, demonstrating that sampling improves the ceiling of TopK, which is impossible for a search-only algorithm.

6 Conclusion

In this work, we first present the limitation of TopK attention approximation for addressing the computational and memory challenges of long-context LLM generation. Then we show oracle sampling can go beyond TopK and introduce MAGICPIG, a novel approach that leverages LSH sampling to approximate the oracle sampling. MAGICPIG significantly reduces the workload of attention computation while preserving high accuracy across diverse tasks. MAGICPIG relies on LSH sampling and a system co-design that offloads hash tables and reduced attention computation to the CPU. Our experimental results demonstrate that MAGICPIG substantially improves throughput and latency across multiple hardware configurations, outperforming traditional TopK attention mechanisms. The theoretical soundness, robustness, and scalability of MAGICPIG open up new opportunities in both attention approximation methods and algorithm-hardware co-design.